From counterexamples to proofs

A tutorial on solver-aided reasoning in Haskell with SBV

LEVENT ERKÖK

Abstract

How can functional programmers apply automated reasoning techniques while keeping programs, specifications, and proofs in the same programming language and development environment? This tutorial develops a method of solver-aided reasoning in Haskell with SBV. The method rests on three decisions: how to formulate a question so that its answer means what we intend, how to represent programs and data so that a solver can reason about them, and how to structure an argument when automation alone is not enough. Counterexamples refine specifications, the same models support search, optimization, and synthesis, and induction and invariants extend proofs from simple properties to recursive and stateful programs. Recurring examples connect these ideas; a binary-search overflow found as a counterexample, for instance, later becomes part of a correctness proof. We close by examining what a successful check establishes, from generated code to the limits of the model. No background in automated reasoning is assumed, and an executable companion supplies checked examples and exercises.

Contents

1 Introduction 3

I. Formulating questions

2 Semantic domains and counterexamples 5
2.1 Symbolic values and their types 5
2.2 Symbolic operators 5
2.3 Validity, counterexamples, and vacuity 6
3 Independent specifications 7
3.1 Saturating arithmetic 7
3.2 A midpoint that leaves its interval 8
4 Bit-vectors and machine-word reasoning 9
4.1 Widths, signedness, and modular arithmetic 9
4.2 Masks, shifts, rotations, and bit fields 9
4.3 Widening, narrowing, and checked arithmetic 9
4.4 Bit tricks and independent specifications 9
5 Exact and floating-point arithmetic 9
5.1 Rationals and algebraic reals 9
5.2 Rounding modes and the limits of floating-point identities 9
6 Constraint solving and model enumeration 9
6.1 Satisfying models and finite enumeration 9
7 Optimization and competing objectives 9
7.1 Soft constraints, weights, and objective priorities 11

II. Choosing representations

8 Host evaluation and symbolic structure 11
8.1 Haskell lists of symbolic values 11
8.2 Symbolic lists and recursive functions 11
8.3 Circuit equivalence through host-language recursion 12
9 Incremental solver interaction 12
9.1 Scoped assumptions and solver state 14

9.2 Unsatisfiable cores and explanations of failure 14
10 Abstraction through contracts 14
10.1 Uninterpreted sorts and functions 14
10.2 Explicit assumptions and consistent contracts 14
11 Symbolic datatypes 14
11.1 Optional results and partial operations 15
12 Arrays and symbolic memory 15
12.1 Read-after-write laws and aliasing 15
12.2 Bounds and total memory models 15
13 Constraints over strings and sets 16
13.1 Regular expressions as specifications 16
13.2 Set membership and representation invariants 16
14 Quantifiers and program synthesis 16
14.1 Quantifier order and witness dependence 16
14.2 Expression search from examples 16
14.3 Counterexample-guided expression synthesis 16

III. Structuring arguments

15 Proofs by induction 16
15.1 A first inductive proof 16
15.2 Solver obligations in a calculation 17
15.3 Generalization over accumulators 17
16 Kadane's algorithm and invariant discovery 19
16.1 Specification 19
16.2 Implementation 19
16.3 Invariant discovery 20
16.4 The proof 20
16.5 Lessons 22
17 Induction on decreasing measures 22
17.1 Expression trees 22
17.2 Binary search with a decreasing interval 23
18 State transitions and inductive invariants 24
18.1 Transition systems and bounded model checking 24
18.2 Inductive invariants and strengthening 24
18.3 Weakest preconditions and imperative programs 24
19 Polymorphic and higher-order proofs 24
19.1 Polymorphic scripts, monomorphic proofs 24
19.2 The bookkeeping law 25
19.3 An arbitrary operation at a fixed type 26
19.4 Closures and higher-order functions 27
20 A verified tautology checker 27

IV. Boundaries and context

21 C generation from verified functions 29
21.1 A verified function across the translation boundary 29
21.2 Executable validation and trust assumptions 29
22 The execution model and its proof obligations 30
22.1 Termination is a proof obligation 31
22.2 Finite lists and datatypes 31
23 Proof results, diagnostics, and trust 31
23.1 Failed checks and diagnostic steps 31
23.2 Encodings and solver selection 32

23.3 Model validation and proof dependencies 32
24 Connections to program reasoning 32
24.1 Testing and symbolic evaluation 32
24.2 Equational reasoning and proof guidance 33
24.3 Refinements, invariants, and the scope of a proof 33
25 Conclusion 34
Acknowledgments 34
References 34

1 Introduction

Haskell programmers often express the behavior of their code as properties. With QuickCheck (Claessen and Hughes 2000), these can be ordinary functions returning `Bool`; a test campaign searches for inputs that falsify them. SBV (Erkök 2026) lets the same programmer ask a stronger question. A property over *symbolic* values is translated to logic and handed to a solver for satisfiability modulo theories (SMT). Such a solver combines logical reasoning with theories such as integer arithmetic and fixed-width bit-vectors. When it can settle the query, it proves the property for every input or produces a concrete counterexample:

```
ghci> prove $ \x -> abs (x :: SInteger) .>= 0
Q.E.D.
ghci> prove $ \x -> abs (x :: SInt8) .>= 0
Falsifiable. Counter-example:
  s0 = -128 :: Int8
```

The two properties differ only in a type. Over mathematical integers, absolute values are nonnegative. Over signed bytes, 128 is not representable, so `abs (-128)` is −128. The solver did not test this value by luck: it solved a constraint whose domain contains all 256 inputs and reported a value that fails.

Our aim is to make this feedback part of ordinary development, with programs, specifications, and proofs expressed in the same programming language and development environment. The tasks extend beyond checking a property: constraints can describe a counterexample, a feasible solution, or a candidate program, and objectives can rank the solutions. A single solver call may settle an arithmetic property, while a recursive program may require a stronger statement, an induction hypothesis, or intermediate equalities. These activities share the three decisions developed throughout the tutorial: formulate a question whose answer has the intended meaning, choose a representation the solver can reason about, and supply an argument when automation alone is insufficient.

Functional programming provides useful structure for these decisions. Pure functions admit equational specifications; recursive definitions suggest induction principles; and higher-order combinators expose the algebraic laws on which a proof depends. SBV keeps programs, specifications, and proof scripts executable in Haskell while constructing solver queries from symbolic definitions. This reduces the need to translate definitions between tools and reconcile their syntactic and semantic differences by hand. It also lets counterexamples become inputs for ordinary evaluation, linking solver feedback back to the program. The choice of symbolic domain determines what the resulting proof establishes.

The tutorial assumes familiarity with Haskell's algebraic datatypes, type classes, and monads, but no knowledge of SMT or theorem proving. It is intended for functional programmers and advanced students learning to mechanize program reasoning. The first three parts develop the three decisions; a fourth examines their boundaries and context:

- *Formulating questions.* Sections 2–7 begin with semantic domains and independent specifications. Mathematical integers, machine words, and floating-point arithmetic give different meanings to apparently similar properties. Saturating addition and a binary-search midpoint illustrate how counterexamples distinguish intended behavior from implementation details. Constraint solving and optimization turn specifications into search for feasible and preferred solutions, separating hard requirements from preferences.
- *Choosing representations.* Sections 8–14 distinguish Haskell evaluation from symbolic structure, with circuit equivalence as an example. Incremental queries organize repeated solver calls, and unsatisfiable cores help explain contradictory assumptions. Uninterpreted functions support abstraction through contracts; datatypes, arrays, strings, and sets provide different models of data. Quantifier order distinguishes finding one program that works for every input from finding an answer for each input. Expression search develops into synthesis by checking candidates and using counterexamples to refine the next search.
- *Structuring arguments.* Sections 15–20 combine equational calculations with induction. Generalization supplies the hypotheses needed for accumulator programs, including list reversal and Kadane's algorithm. Decreasing measures support proofs about expression trees and binary search; invariant strengthening carries the same lesson into stateful programs. Polymorphic and higher-order proofs extend the method to reusable abstractions, and a verified tautology checker brings these techniques into a larger development.
- *Boundaries and context.* Sections 21–24 follow a verified function through C generation, then examine the execution model, termination obligations, and the scope of finite-data models. Diagnostics, solver selection, and proof dependencies help interpret successful and unsuccessful checks; connections to other approaches place the method in context.

Recurring examples connect the parts: the midpoint counterexample returns in the binary-search proof, expression trees connect search and synthesis to induction, and accumulator generalization becomes invariant strengthening for state transitions. Readers primarily interested in induction can proceed from Sections 2–3 and 8 to Section 15, returning to contracts, datatypes, and arrays as needed.

We verify functions expressed with SBV's symbolic operations; SBV does not translate arbitrary existing Haskell source into logic. In particular, a proof about mathematical integers or finite lists must not be silently transferred to machine integers, partial computations, or infinite lists. Making these choices explicit is part of the method, alongside writing and checking the proof.

Following along. The standalone companion pins its dependencies, including an SBV 14.10 source snapshot, and declares the required language extensions. It was checked with GHC 9.14.1, Z3 5.3.2 (de Moura and Bjørner 2008), and CVC5 1.4.2 (Barbosa et al. 2022). Install the solvers on `PATH`; Cabal does not install them. From the companion directory, build the examples and start GHCi:

```
cabal build tutorial-examples
cabal repl tutorial-examples
```

The session loads all imports, including `Data.SBV.List` qualified as `SL`, and enables the extensions used below. Run the checks with `cabal run tutorial-examples`; the README gives full validation instructions. The execution log records source hashes and exact tool versions. Each exercise names its checked solution in the companion.

Listings are extracted from checked sources; transcripts and elisions are marked separately. Model values and their order can vary between solver versions, so validation checks their meaning rather than their printed form.

2 Semantic domains and counterexamples

2.1 *Symbolic values and their types*

SBV's central type is `SBV a`, a symbolic value whose concrete counterpart has type `a`. Abbreviations name the common cases: `SBool`, `SInteger` (unbounded integers), `SWord8` through `SWord64` and `SInt8` through `SInt64` (bit-vectors with wrap-around arithmetic), `SReal` (exact real numbers), `SFloat` and `SDouble` (IEEE-754), `SString`, and `SList a`. There are also arrays, sets, tuples, `Maybe`, `Either`, and user-defined algebraic datatypes.

These types are not interchangeable encodings of one numeric type; each is a different mathematical problem. The introduction showed that the choice between `SInteger` and `SInt8` changes a theorem into a false statement. Floating point has its own surprises:

```
ghci> prove $ \(x :: SFloat) -> x .== x
Falsifiable. Counter-example:
  s0 = NaN :: Float
```

Equality in IEEE-754 is not reflexive, and the solver knows it. Choosing a symbolic type is choosing the semantics the solver will use, so it deserves the same care as choosing a type for a production program.

2.2 *Symbolic operators*

Arithmetic on symbolic values uses the ordinary `Num` class, so `x + 1` and `2 * y` work as expected. Comparisons cannot reuse `Eq` and `Ord`: their methods return a Haskell `Bool`, which must be known when Haskell evaluates it, while the result of comparing symbolic values may be unknown. SBV therefore provides `.==`, `./=`, `.<`, `.<=`, `.>`, and `.>=`, which return `SBool`. Boolean connectives are `.&&`, `.||`, `sNot`, and implication `.=>`.

For the same reason, Haskell's `if-then-else` cannot branch on an `SBool`. The function `ite :: Mergeable a => SBool -> a -> a -> a` builds a symbolic choice that the solver resolves. This distinction between *Haskell control* and *symbolic control* recurs throughout the tutorial. Concrete arguments allow symbolic operations to compute immediately. For example, `ite sTrue 10 20 :: SInteger` produces the literal `10`; no solver is involved.

Table 1. Entry points into SBV. All of them consume the same symbolic programs.

Interface	You supply	Successful result	Section
`prove`	A property	Validity or a counterexample	2–3
`sat`, `allSat`	Constraints on unknowns	One or all satisfying models	6
`optimize`	Constraints and objectives	Optimal models or objective bounds	7
`query`	A Haskell program driving the solver	Whatever the program computes	9
TP	Lemmas, induction, calculations	Checked proofs with dependencies	15–20
`compileToC`	A symbolic function	C source code	21

Solver calls can also be inconclusive. Enumeration can fail to terminate, and an optimization objective need not have an attained optimum.

2.3 *Validity, counterexamples, and vacuity*

Calling `prove` on a function of symbolic arguments creates a fresh unknown for each argument, evaluates the function to obtain a symbolic formula $P(\vec{x})$, and asks the solver whether $\neg P(\vec{x})$ is satisfiable. If it is not, P holds for every input and SBV prints `Q.E.D.` If it is, the satisfying assignment is a counterexample. A third outcome is possible: for undecidable or very hard problems, the solver may give up or time out. SBV reports this as an unknown result, which is neither a proof nor a refutation.

Unknowns can also be created by name inside the `Symbolic` monad, which gives readable counterexamples and allows assumptions:

```
prove $ do x <- sInteger "x"
           y <- sInteger "y"
           constrain $ x .> 0 .&& y .> 0
           pure $ x * y .> 0
```

Here `constrain` restricts attention to inputs that satisfy the assumption. A contradictory assumption makes every property provable, so assumptions should be kept as weak as the property permits. For assumptions A and conclusion P, `prove` searches for a model of $A \wedge \neg P$; it does not establish that A is feasible. For example,

```
ghci> prove $ \(x :: SWord8) -> x .> 255 .=> x .== 0
Q.E.D.
ghci> sat $ \(x :: SWord8) -> x .> 255
Unsatisfiable
```

The first answer is a vacuous proof: no byte satisfies its premise. Checking the premise with `sat` exposes the modeling mistake. Here and below, fresh arguments have the default universal interpretation under `prove` and existential interpretation under `sat`. Explicit mixed quantifiers are possible but outside this tutorial.

Table 1 lists the main ways of using a symbolic model. The rest of the tutorial visits each of them.

3 Independent specifications

A proof is only as good as the property it proves. The most useful properties compare an implementation with a specification written in a different, simpler way.

3.1 Saturating arithmetic

Consider addition on unsigned bytes that saturates at 255 instead of wrapping. An implementation can avoid overflow by testing before it adds, while the specification can use unbounded integers, where overflow cannot happen:

```
satAdd :: SWord8 -> SWord8 -> SWord8
satAdd x y = ite (x .> 255 - y) 255 (x + y)

satAddSpec :: SWord8 -> SWord8 -> SBool
satAddSpec x y = sFromIntegral (satAdd x y) .== ite (total .> 255) 255
    total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

On concrete inputs, `satAdd 250 10` evaluates to 255 without a solver. The guard `x .> 255 - y` cannot overflow, because `255 - y` is representable for every byte `y`, and on the other branch `x + y` does not wrap. In the specification, `sFromIntegral` converts each byte to an integer *before* adding. Converting the already wrapped byte sum would repeat the implementation's arithmetic and could hide its mistakes.

```
ghci> prove satAddSpec
Q.E.D.
```

The proof covers all 65,536 input pairs. A tempting alternative implementation shows why the independent specification matters:

```
wrongAdd :: SWord8 -> SWord8 -> SWord8
wrongAdd x y = ite (x + y .> 255) 255 (x + y)

wrongAddSpec :: SWord8 -> SWord8 -> SBool
wrongAddSpec x y = sFromIntegral (wrongAdd x y) .== ite (total .> 255) 255
    total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

```
ghci> prove $ do { x <- sWord8 "x"; y <- sWord8 "y"; pure (wrongAddSpec x y
    ) }
Falsifiable. Counter-example:
  x = 223 :: Word8
  y = 124 :: Word8
```

The sum 223 + 124 = 347 wraps to 91, and no byte is greater than 255, so the saturation branch is never taken. Had the specification also used byte arithmetic, it would have agreed with the bug. Your solver may report a different pair; any pair whose sum exceeds 255 is a valid counterexample, and the companion checks the reported pair with ordinary Haskell arithmetic.

Exercise 1. Write `satSub :: SWord8 -> SWord8 -> SWord8`, subtraction that saturates at 0, together with a specification over `SInteger`, and prove that they agree. Solutions: `satSub` and `satSubSpec`.

3.2 A midpoint that leaves its interval

Binary search supplies another example of why the arithmetic domain matters. The midpoint bug described by Bloch (2006) arises when `low + high` overflows before division by two. We can expose it without building an array: the local arithmetic already violates the intended bounds. The following functions, adapted from SBV's `BitPrecise.BrokenSearch` example, use signed 32-bit indices:

```
midPointBroken, midPointFixed :: SInt32 -> SInt32 -> SInt32
midPointBroken low high = (low +! high) /! 2
midPointFixed  low high = low +! ((high -! low) /! 2)
```

The operators `+!`, `-!`, and `/!`, from `Data.SBV.Tools.Overflow`, attach safety assertions to the arithmetic. The function `safe` checks these assertions under the supplied constraints and returns a result for each check:

```
midPointSafety :: (SInt32 -> SInt32 -> SInt32) -> IO [SafeResult]
midPointSafety f = safe $ do
  low  <- sInt32 "low"
  high <- sInt32 "high"
  constrain $ 0 .<= low .&& low .<= high
  output $ f low high
```

Running `midPointSafety midPointBroken` reports an addition overflow and a counterexample. For a simple instance, take $low = high = 2^{30}$. The sum wraps to -2^{31} and the computed midpoint is -2^{30}, outside the singleton interval. The companion checks both this instance and the solver's reported counterexample using ordinary Haskell arithmetic.

The repair computes the distance before adding half of it to the lower bound. Under $0 \le low \le high$, this avoids an overflowing intermediate sum. We can check all its arithmetic assertions:

```
ghci> all isSafe <$> midPointSafety midPointFixed
True
```

But an overflow-free expression could still compute the wrong answer. We need a separate specification. Signed 64-bit arithmetic can represent the sum of any two signed 32-bit inputs, so it gives a reference calculation:

```
midPointSpec :: (SInt32 -> SInt32 -> SInt32)
             -> SInt32 -> SInt32 -> SBool
midPointSpec f low high = 0 .<= low .&& low .<= high .=>
  sFromIntegral (f low high) .== (low64 + high64) `sDiv` 2
  where low64  = sFromIntegral low  :: SInt64
        high64 = sFromIntegral high :: SInt64
```

```
ghci> prove (midPointSpec midPointFixed)
Q.E.D.
```

As with saturating addition, both inputs are widened *before* the addition. These checks establish arithmetic safety and the value of the repaired midpoint; they do not yet show that binary search finds a target. We return to that algorithmic question in Section 17.2, after introducing arrays and induction.

4 Bit-vectors and machine-word reasoning

4.1 Widths, signedness, and modular arithmetic

4.2 Masks, shifts, rotations, and bit fields

4.3 Widening, narrowing, and checked arithmetic

4.4 Bit tricks and independent specifications

5 Exact and floating-point arithmetic

5.1 Rationals and algebraic reals

5.2 Rounding modes and the limits of floating-point identities

6 Constraint solving and model enumeration

6.1 Satisfying models and finite enumeration

Proving a property asks whether every input satisfies it. The dual question asks whether *some* input satisfies a constraint, and which one. The function `sat` returns one satisfying model, and `allSat` enumerates all of them. Which bytes are unchanged by squaring?

```
idempotentBytes :: IO AllSatResult
idempotentBytes = allSat $ \(x :: SWord8) -> x * x .== x
```

```
ghci> idempotentBytes
Solution #1:
  s0 = 1 :: Word8
Solution #2:
  s0 = 0 :: Word8
Found 2 different solutions.
```

The answer is a small theorem in disguise: $x(x-1) \equiv 0 \pmod{256}$, and since x and $x-1$ are coprime and $256 = 2^8$, one of them must be divisible by 256. `allSat` repeatedly finds a model and excludes it before searching again. The finite byte domain makes a complete enumeration possible here; on an unbounded domain, asking for all models may never finish.

7 Optimization and competing objectives

A satisfying model witnesses feasibility; an optimization result also describes how a feasible choice compares with alternatives. This adds a modeling decision: which quantities define a better answer, and how should competing objectives be ordered? The function `optimize` ranks feasible solutions by named objective *measures*: `minimize` a cost or `maximize` a profit. SBV's `Metric` class supports symbolic integers, reals, and bit-vectors, among others. These objectives differ from the decreasing measures used later to establish termination. A workshop makes two products with profits 2 and 3, subject to two resource limits:

```
production :: IO OptimizeResult
production = optimize Lexicographic $ do
  x <- sInteger "x"
  y <- sInteger "y"
  constrain $ x .>= 0 .&& y .>= 0
  constrain $ x + 2 * y .<= 10
  constrain $ 3 * x + y .<= 15
  maximize "profit" (2 * x + 3 * y)
```

Running `production` gives $x = 4$, $y = 3$, and profit 17. Here `constrain` sets hard requirements. Over `SInteger` this is integer linear programming; using `SReal` instead allows fractional production quantities. The resource bounds ensure that the maximum is attained. In general, check for infeasibility, unbounded objectives, and unattained bounds: maximizing a real x subject to $x < 1$ has supremum 1 but no optimal model.

Combining measures. The first argument of `optimize` selects an `OptimizeStyle`:

- `Lexicographic` optimizes objectives in declaration order. Once the first objective is optimal, the second breaks ties, and so on. For example, maximize profit first, then minimize the number of units produced without sacrificing profit.
- `Independent` uses Z3's *box* mode: optimize each objective separately under the same constraints, returning a model for each. These individual optima need not be jointly attainable.
- `Pareto Nothing` enumerates nondominated objective vectors: improving one objective requires worsening another. Their set is the *Pareto front*. Use `Pareto (Just n)` to request at most n such points; an infinite front makes unrestricted enumeration nonterminating.

For comparison, maximize nonnegative integers x and y with $x + y \leq 10$. Lexicographic order x then y gives $(10, 0)$; box gives separate maxima $x = 10$ and $y = 10$; the Pareto front consists of the eleven pairs with $x + y = 10$. Box does not promise the infeasible pair $(10, 10)$.

Bounds in an extension field. Extension-field results apply to both `SInteger` and `SReal`: unbounded objectives can yield `oo` or `-oo`. Real objectives can additionally require infinitesimals. For example, independently minimize and maximize a positive real:

```
realBounds :: IO OptimizeResult
realBounds = optimize Independent $ do
  x <- sReal "x"
  constrain $ x .> 0
  minimize "lower" x
  maximize "upper" x
```

Both independent results report these extension-field bounds in the tested configuration:

```
lower = epsilon :: Real
upper =      oo :: Real
```

Here `epsilon` denotes a positive infinitesimal: the infimum is 0 but is unattained. It is not a small floating-point tolerance. The symbol `oo` denotes positive infinity: the maximum is unbounded. Neither is a concrete real assignment to `x`. SBV exposes such bounds through `getModelObjectives` as `GeneralizedCV` values, using `ExtendedCV` for extension-field

expressions. Changing `sReal` to `sInteger` in this example makes the minimum 1, while the maximum remains `oo`.

Strict-bound pitfall. Infinitesimal results are not reliable for every formulation or Z3 version: issue 5314 reports this minimization returning 1 without an unknown result (Erkök and Bjørner 2021). Check strict-bound behavior; replacing $>$ by $\geq$ changes the problem.

Solver support and arithmetic limits. Z3 is currently the only optimization backend supported by SBV. SMT-LIB has not standardized optimization commands; SBV uses Z3's extensions for objectives and priorities (Microsoft ndb). Optimization is therefore less portable across solvers than ordinary satisfiability checking.

Products of unknowns, such as `x * y`, introduce nonlinear arithmetic, which can be difficult even without optimization; over integers, satisfiability is undecidable (Microsoft nda). Z3's polynomial real procedures include optimization support since 5.3, but do not provide a general solution for nonlinear integer or mixed problems (Microsoft ndb). Prefer faithful linear formulations and distinguish feasible candidates from established optima.

Exercise 2. With `production`'s constraints, maximize `x + 2*y`, then minimize `x + y` lexicographically. Compare with the opposite priority order. Solutions: `productionLex` for the original order and `productionLexReversed` for the reverse.

7.1 Soft constraints, weights, and objective priorities

8 Host evaluation and symbolic structure

An SBV interaction separates two kinds of work: Haskell evaluates the program to build a symbolic formula, and the solver reasons about that formula. Query programs can alternate between these activities. Knowing which side handles which part of a program explains how recursion behaves.

8.1 Haskell lists of symbolic values

A Haskell list `[SInteger]` has a spine that Haskell owns and elements that may be symbolic. Functions such as `sum`, `map`, and `foldr` work on it unchanged, because they only rearrange the spine and apply `Num` operations to the elements:

```
nonNegSum :: Int -> Predicate
nonNegSum n = do
  xs <- mapM (\i -> sInteger ("x" ++ show i)) [1 .. n]
  pure $ sAll (.>= 0) xs .=> sum xs .>= 0
```

For example, `prove (nonNegSum 5)` establishes the property for five symbolic integers. Haskell unfolds `sum` into `x1 + x2 + x3 + x4 + x5` before the solver sees anything; the recursion has disappeared. This style is ideal for fixed-size problems: circuits, unrolled loops, puzzles on fixed boards. The price is that the theorem is about lists of length five, not all lists.

8.2 Symbolic lists and recursive functions

The type `SList Integer` denotes a *finite* sequence whose length and contents may both be unknown. There is no fixed length bound, but infinite lists and partial values are outside this domain. Throughout the tutorial, a theorem about "every list" means every finite list of values in the modeled element type (Section 22.2). To state such a property, we need a

function whose recursion is part of the logic, not something Haskell unfolds. Writing a naive recursive Haskell function over `SList` does not work: whether the list is empty is unknown, so Haskell would unfold both branches forever. SBV's `smtFunction` solves this problem. It gives the function a name, and recursive calls become calls to that named function in the logic:

```
sumL :: SList Integer -> SInteger
sumL = smtFunction "sumL" $ \xs -> [sCase| xs of
          []     -> 0
          y : ys -> y + sumL ys
       |]
```

The `sCase` quasi-quoter provides pattern matching on symbolic values. The definition reads like ordinary Haskell, but SBV translates it to a single recursive SMT-LIB definition (Barrett et al. 2025). Because `sumL` is a function of symbolic values, it still computes on literals: `sumL [1,2,3]` evaluates to `6` without a solver. When a recursive definition is registered for symbolic reasoning, SBV checks termination; for `sumL`, it automatically finds the list length as a decreasing, nonnegative measure. Termination makes the definition admissible; it does not prove arbitrary properties of it.

Now try the obvious generalization of `nonNegSum`:

```
z3Timeout5 :: SMTConfig
z3Timeout5 = z3 { solverSetOptions = solverSetOptions z3 ++ [SetTimeOut
    5000] }

sumWithoutInduction :: IO ThmResult
sumWithoutInduction =
  proveWith z3Timeout5 $
    \xs -> SL.all (.>= 0) xs .=> sumL xs .>= 0
```

The property is true, but ordinary SMT reasoning about recursive definitions does not reliably establish it. The configuration preserves Z3's default options and limits each solver call to five seconds; an unknown answer is expected with the tested configuration. It says nothing about the truth of the property. Section 15 supplies an induction argument that works for lists of every finite length.

The general rule is: *recursion over Haskell structure happens while building the problem; recursion through `smtFunction` becomes part of the problem*. The former fixes the amount of unfolding before solving; the latter allows statements about structures of unbounded size. Neither representation guarantees that the resulting solver query will be easy.

8.3 Circuit equivalence through host-language recursion

9 Incremental solver interaction

The functions `prove`, `sat`, and `optimize` manage solver interactions on the caller's behalf. Many applications need a conversation: ask, inspect the answer, and ask a refined question. SBV's `query` mode hands control of a live solver session to a Haskell program. The following function finds the smallest integer greater than 10 that leaves remainder 3 when divided by 7 and remainder 4 when divided by 5:

```
smallest :: IO Integer
```

```
smallest = runSMT $ do
  x <- sInteger "x"
  constrain $ x .> 10
  constrain $ x `sMod` 7 .== 3
  constrain $ x `sMod` 5 .== 4
  query $ let descend best = do
                r <- checkSat
                case r of
                  Sat   -> do v <- getValue x
                              constrain $ x .< literal v
                              descend (Just v)
                  Unsat -> maybe (io $ fail "no solution") pure best
                  _     -> io $ fail "inconclusive"
          in descend Nothing
```

```
ghci> smallest
24
```

The constraints before `query` set up the problem as usual. Inside the query, `checkSat` asks for a model, and `getValue` reads the value of `x` in that model. That value is an ordinary `Integer`, so Haskell can branch on it and use it to build the next constraint, `x .< literal v`, which asks for a smaller solution. When no smaller solution exists, the last model found is the minimum. The `case` on the solver's answer is Haskell control; the constraint that it adds is symbolic. Keeping the two apart is what makes the solver usable as a component of a larger algorithm.

Three properties of this loop matter. First, successful candidates are strictly decreasing *integers* bounded below by 11. After the first candidate v, there can be at most $v - 11$ further decreases. Thus the loop terminates provided each solver call returns. The existence of a minimum alone would not justify this reasoning over reals: the candidates could decrease forever without reaching it. Second, the solver may return the candidates in any order. Our loop does not depend on that order; it only relies on unsatisfiability at the end. Third, the loop handles an unknown answer separately: an inconclusive solver response is not evidence that no smaller value exists.

Queries can do much more: `push` and `pop` add and retract temporary assumptions, named constraints allow unsatisfiable cores to be extracted, and new unknowns can be created in the middle of a session with `freshVar`. The last point is less trivial than it looks: a new variable may have a type, such as a user-defined datatype, that the solver has not seen yet. SBV tracks the declarations each new term needs and sends them before the term is used, so queries compose without regard to the order in which types are first used.

Exercise 3. Change `smallest` to find the *largest* solution below 100. (The answer is 94.) Solution: `largestBelow100`.

9.1 Scoped assumptions and solver state

9.2 Unsatisfiable cores and explanations of failure

10 Abstraction through contracts

10.1 Uninterpreted sorts and functions

10.2 Explicit assumptions and consistent contracts

11 Symbolic datatypes

A datatype declaration can serve two roles in a solver-aided program: it defines the candidates a search may return, and the structures over which a theorem may quantify. We use one expression language for both roles, first synthesizing an expression and later proving that a transformation preserves its meaning (Section 17). User-defined algebraic datatypes become symbolic with one line of Template Haskell:

```
data Expr = Num Integer
          | Add Expr Expr
          | Mul Expr Expr
          deriving Show
mkSymbolic [''Expr]

eval :: SExpr -> SInteger
eval = smtFunction "eval" $ \e -> [sCase| e of
          Num n   -> n
          Add a b -> eval a + eval b
          Mul a b -> eval a * eval b
       |]

puzzle :: IO SatResult
puzzle = sat $ \e -> eval e .== 42 .&& isMul e .&& sNot (isNum (getMul_1 e)
    )
```

The call `mkSymbolic [''Expr]` generates the type `SExpr`, symbolic constructors (`sNum`, `sAdd`, `sMul`), recognizers (`isNum`, `isAdd`, `isMul`), and field accessors such as `getMul_1`. The evaluator is an `smtFunction`, so it is a recursive definition in the logic, and `sCase` matches on an expression whose constructor may be unknown. The puzzle asks the solver to *synthesize* an expression: a multiplication whose left argument is not a number, evaluating to 42.

```
ghci> puzzle
Satisfiable. Model:
  s0 = Mul (Mul (Num 1) (Num (-1))) (Num (-42)) :: Expr
```

The model is an ordinary Haskell value of type `Expr`, which we can inspect, print, or evaluate with a separate Haskell interpreter. The companion does exactly that to check the answer independently of the solver. Different solver versions may find different expressions.

The constructor test `isMul e` matters: an accessor such as `getMul_1 e` describes the intended field only when `e` has that constructor. Also, this is synthesis from a structural constraint and one evaluation constraint, not a proof that the expression implements a function

on every input. The search has no explicit depth bound and is not guaranteed to terminate for arbitrary constraints.

The same datatype can be the subject of proofs: a property of `eval` over all expressions concerns finite expression trees of unbounded size, and may need induction. Infinite or partial trees are outside the model (Section 22.2). Section 17 returns to `Expr` with a proof that a recursive transformation preserves evaluation.

11.1 Optional results and partial operations

12 Arrays and symbolic memory

The expression trees above describe data by constructors. Memory is often more naturally described by what can be observed at an address. SMT arrays support this second representation: proofs use read-after-write laws and equality of mappings, rather than induction over a sequence of stored cells. An `SArray Integer Integer` is a total mapping from integer addresses to integer values. Use `sArray "mem"` to create an unknown array and `readArray mem i` to read a symbolic address. Updates are functional: `writeArray mem i v` returns an updated array, preserving `mem` as the previous state.

12.1 Read-after-write laws and aliasing

The array theory supplies read-after-write laws: the updated address returns the new value; all other addresses retain their values. Equality is *extensional*: equal arrays agree at every address, and conversely (Tinelli 2024). Thus writes to distinct addresses commute:

```
writesCommute :: SArray Integer Integer
              -> SInteger -> SInteger -> SInteger -> SInteger -> SBool
writesCommute a i j x y =
  i ./= j .=>
    writeArray (writeArray a i x) j y .==
    writeArray (writeArray a j y) i x
```

```
ghci> prove writesCommute
Q.E.D.
```

The proof covers every initial array, both addresses, and both values. The solver establishes equality of entire arrays directly, without traversal or induction.

Omit `i ./= j` and the property is false: the addresses may *alias*. For example, with $i = j = 0$, $x = 1$, and $y = 2$, the left side stores 2 at address 0 while the right side stores 1. The last write wins, regardless of the initial contents. The companion obtains a counterexample from the solver and checks its updates independently using Haskell association-list lookup.

12.2 Bounds and total memory models

SMT arrays have no built-in length or out-of-bounds error. For a buffer of length `n`, prove `0 .<= i .&& i .< n` at each access; assuming the bound would exclude unsafe accesses. `constArray v` initializes every address to `v`, while `lambdaArray` defines values by index, for example to update an interval and preserve `readArray mem i` elsewhere. The index type determines the domain: `Word8` has 256 addresses and `Integer` infinitely many. These total mappings differ from the finite recursive structure of lists and datatypes (Section 22.2).

Exercise 4. Prove that writing twice to the same address equals performing only the last write, for every array, address, and pair of values. Solution: `lastWriteWins`.

13 Constraints over strings and sets

13.1 Regular expressions as specifications

13.2 Set membership and representation invariants

14 Quantifiers and program synthesis

14.1 Quantifier order and witness dependence

14.2 Expression search from examples

14.3 Counterexample-guided expression synthesis

15 Proofs by induction

SMT solvers decide many theories well, but an induction argument may be needed to connect recursive definitions to a theorem. SBV's TP layer (in `Data.SBV.TP`) lets the programmer write the argument in Haskell and has the solver check each step. The programmer decides *what* to prove and *how* to break it into steps; the solver decides whether each step is valid.

This division connects symbolic execution to calculational reasoning. We choose intermediate expressions and useful hypotheses; the solver checks the logical consequences of each step. The notation below makes the proof obligations explicit. Section 24.2 relates this style to equational reasoning in proof assistants and refinement-based proofs.

15.1 A first inductive proof

We return to the property that eluded `prove` in Section 8:

```
sumNonNeg :: TP (Proof (Forall "xs" [Integer] -> SBool))
sumNonNeg = induct "sumNonNeg"
  (\(Forall xs) -> SL.all (.>= 0) xs .=> sumL xs .>= 0) $
  \ih (x, xs) -> [SL.all (.>= 0) (x .: xs)]
              |- sumL (x .: xs) .>= 0
              =: x + sumL xs .>= 0
              ?? ih
              =: sTrue
              =: qed
```

The type `Proof (Forall "xs" [Integer] -> SBool)` records quantification over integer lists; the predicate is supplied to `induct`. The name `xs` identifies the variable for output and instantiation. Scripts live in the `TP` monad: `runTP` checks their obligations and returns a proof object; Haskell typechecking alone does not establish the theorem.

The function `induct` applies list induction: SBV checks the base case automatically, then supplies the hypothesis `ih` for `xs` to prove the property for `x .: xs`. This covers finite lists; the scope for lists and recursive datatypes is discussed in Section 22.2.

The step is a *calculation*. Its first line lists the assumptions, here the precondition for the longer list, and `|-` starts a chain of expressions connected by `=:`. Each link must be justified by the definitions, the assumptions, and any hints supplied with `??`. The chain ends with `qed`. A hint written between two expressions justifies the transition *to the following expression*; `?? ih` is not a new assumption about the entire theorem. In this example the expressions

being equated are Boolean formulas, and the calculation reduces the target formula to `sTrue`. Running the proof shows each checked step (the final proof-object display is omitted here):

```
ghci> runTP sumNonNeg
Inductive lemma: sumNonNeg
  Step: Base                 Q.E.D.
  Step: 1                    Q.E.D.
  Step: 2                    Q.E.D.
  Result:                    Q.E.D.
Functions proven terminating: sbv.foldr, sumL
```

Step 1 unfolds `sumL` once. Step 2 uses the hypothesis: `x` is nonnegative by assumption and so is `sumL xs`, so the sum is too. The `Result` line checks that the chain, together with the base case, establishes the theorem as stated. The last line reports that the recursive functions used, including the library's `foldr` that underlies `SL.all`, were checked for termination. Section 22.1 explains how TP enforces this requirement.

15.2 Solver obligations in a calculation

Figure 1 shows the queries under assumptions A, with facts H_i supplied by hints h_i. Definitions are available in every query. SBV also checks that the chain establishes the stated theorem under its precondition. For failed steps, see Section 23.1.

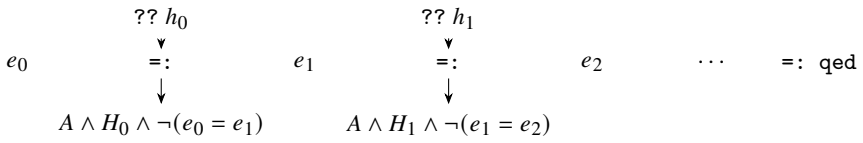


Fig. 1. Each hint supplies a fact for the next equality; its query must be unsatisfiable.

15.3 Generalization over accumulators

Reversing a list with an accumulator is a standard optimization:

```
revAcc :: SList Integer -> SList Integer -> SList Integer
revAcc = smtFunction "revAcc" $ \acc xs -> [sCase| xs of
            []     -> acc
            x : xt -> revAcc (x .: acc) xt
         |]
```

We expect `revAcc [] xs` to equal `SL.reverse xs`. The direct attempt is instructive:

```
revWeak :: TP (Proof (Forall "xs" [Integer] -> SBool))
revWeak = induct "revWeak"
  (\(Forall xs) -> revAcc [] xs .== SL.reverse xs) $
  \ih (x, xs) -> [] |- revAcc [] (x .: xs)
                    =: revAcc [x] xs
                    ?? ih
                    =: SL.reverse xs SL.++ [x]
                    =: SL.reverse (x .: xs)
                    =: qed
```

Run it with a five-second per-call timeout (intermediate output omitted):

```
ghci> runTPWith z3Timeout5 revWeak
...
*** Failed to prove revWeak.2.
```

Step 2 tries to use the induction hypothesis, but the hypothesis is about `revAcc [] xs`, while the expression contains `revAcc [x] xs`. After one step of recursion, the accumulator is no longer empty, and the hypothesis says nothing about it. The timeout leaves the statement unresolved; it does not refute reversal correctness.

The fix is the oldest trick in inductive proofs: prove a more general statement. The theorem must describe `revAcc` for *every* accumulator, so that the hypothesis can be used at whatever accumulator the recursion produces:

```
revAccCorrect :: TP (Proof (Forall "xs" [Integer] -> Forall "acc" [Integer]
     -> SBool))
revAccCorrect = induct "revAccCorrect"
  (\(Forall xs) (Forall acc) -> revAcc acc xs .== SL.reverse xs SL.++ acc)
    $
  \ih (x, xs) acc -> [] |- revAcc acc (x .: xs)
                        =: revAcc (x .: acc) xs
                        ?? ih
                        =: SL.reverse xs SL.++ (x .: acc)
                        =: (SL.reverse xs SL.++ [x]) SL.++ acc
                        =: SL.reverse (x .: xs) SL.++ acc
                        =: qed
```

Here `acc` is universally quantified, and the induction hypothesis `ih` holds for all accumulators. When it is supplied as a hint, the solver instantiates it with `x .: acc`. The remaining steps use ordinary laws of `++` and `reverse`, using their sequence operations and definitions. The original statement is the special case `acc = []`:

```
reverseCorrect :: TP (Proof (Forall "xs" [Integer] -> SBool))
reverseCorrect = do
  generalized <- recall revAccCorrect
  lemma "reverseCorrect"
    (\(Forall xs) -> revAcc [] xs .== SL.reverse xs)
    [proofOf generalized]
```

`recall` obtains the generalized proof, reusing it if it is already cached in the current TP run. `proofOf` packages that proof as a dependency for `lemma`, which asks the solver to establish the specialization. This separates the reusable inductive result from the interface property callers need.

Exercise 5. Prove that the length of `revAcc acc xs` equals `SL.length xs + SL.length acc`. Why must the accumulator remain universally quantified in the induction hypothesis, even when the intended caller starts with `[]`? Solution: `revAccLength`.

16 Kadane's algorithm and invariant discovery

The maximum segment sum problem asks for the largest sum of a contiguous segment of a list, where the empty segment has sum 0. For `[1, -2, 3, 4, -1, 2]`, the answer is 8, from `[3, 4, -1, 2]`. Kadane's algorithm (Bentley 1984, Algorithm 4) solves it in a single left-to-right pass. Its correctness is not obvious, which makes it a good example of finding and proving an invariant.

16.1 Specification

We specify the problem by its most direct recursive reading:

```
mss :: SList Integer -> SInteger
mss = smtFunction "mss" $ \xs -> [sCase| xs of
         []    -> 0
         _ : t -> mssBegin xs `smax` mss t
      |]

mssBegin :: SList Integer -> SInteger
mssBegin = smtFunction "mssBegin" $ \xs -> [sCase| xs of
              []    -> 0
              h : t -> 0 `smax` (h `smax` (h + mssBegin t))
           |]
```

`mssBegin xs` is the best sum of a segment that starts at the beginning of `xs`: either the empty segment (0), just the head, or the head followed by the best segment starting at the tail. `mss xs` is the best of `mssBegin xs` and the best segment of the tail: a segment either starts at the head or lies entirely in the tail. With unit-cost arithmetic and without memoizing calls across suffixes, these recurrences take quadratic time. Since they are symbolic functions, they also run on literal lists, which is a convenient way to test a specification before reasoning about it: `mss [1, -2, 3, 4, -1, 2]` evaluates to 8.

To check that the recurrence matches the intended problem, we can independently enumerate segments with ordinary Haskell's `Data.List.inits` and `Data.List.tails`:

```
segmentOracle :: [Integer] -> Integer
segmentOracle xs = maximum [sum segment | suffix <- tails xs, segment <-
    inits suffix]
```

Every segment is a prefix of a suffix, and the empty segment ensures the maximum is defined even for `[]`. The companion compares both `mss` and `kadane` against this oracle on all 3,906 lists of length at most five over $\{-2, -1, 0, 1, 2\}$, plus the examples above. These tests check the connection to the intended specification; the theorem below establishes equivalence for every finite integer list.

16.2 Implementation

Kadane's algorithm maintains two accumulators while it scans the list: `meh`, the maximum sum of a segment ending at the current position, and `msf`, the maximum found so far:

```
kadane :: SList Integer -> SInteger
kadane xs = kadaneHelper xs 0 0
```

```
kadaneHelper :: SList Integer -> SInteger -> SInteger -> SInteger
kadaneHelper = smtFunction "kadaneHelper" $ \xs meh msf -> [sCase| xs of
                  []    -> msf
                  h : t -> let meh' = 0 `smax` (h + meh)
                               msf' = msf `smax` meh'
                           in kadaneHelper t meh' msf'
               |]
```

At each element `h`, the best segment ending here either extends the best segment ending at the previous position or is empty, so the new `meh` is `0 `smax` (h + meh)`. The best so far is updated if that value is larger. The allowance of an empty segment matters: an all-negative input returns 0. An API that requires a nonempty segment would need a different specification and initialization.

16.3 Invariant discovery

As with reversal, the theorem we want, `kadane xs .== mss xs`, concerns the initial accumulators `0` and `0`. An inductive proof will reach `kadaneHelper` with other accumulator values, so we must describe what `kadaneHelper xs meh msf` computes for *any* sensible `meh` and `msf`.



Fig. 2. Three candidates for the maximum segment sum: within the processed prefix, within the remaining suffix, or across the boundary. Segments may be empty; each label gives the best sum for that case.

Split the list into a processed prefix and the remaining suffix `xs`. Figure 2 shows the three possible locations of the best segment. The crossing candidate combines the best suffix of the processed prefix (`meh`) with the best prefix of `xs` (`mssBegin xs`).

For example, split `[1,-2,3,4,-1,2]` after the 4. The processed prefix has $meh = 7$ and $msf = 7$, while the suffix has $mss = 2$ and $mssBegin = 1$. The three candidates are 7, 2, and $7 + 1 = 8$; the best segment crosses the boundary. So the invariant is

$$kadaneHelper(xs, meh, msf) = \max\big(msf,\ mss(xs),\ meh + mssBegin(xs)\big),$$

under the preconditions that make the accumulators meaningful: $meh \geq 0$ (the empty segment is always available) and $msf \geq meh$ (the best segment so far is at least as good as the best one ending here). Without the preconditions the statement is false: for an empty `xs`, the right side would include *meh* even when it exceeds *msf*.

16.4 The proof

Once the invariant is written down, the proof is short:

```
kadaneCorrect :: TP (Proof (Forall "xs" [Integer] -> SBool))
kadaneCorrect = do
  inv <- induct "kadaneInvariant"
           (\(Forall xs) (Forall meh) (Forall msf) ->
                    (meh .>= 0 .&& msf .>= meh)
                .=> kadaneHelper xs meh msf
                    .== msf `smax` mss xs `smax` (meh + mssBegin xs)) $
           \ih (a, as) meh msf ->
              [meh .>= 0, msf .>= meh]
           |- let meh' = 0 `smax` (a + meh)
                  msf' = msf `smax` meh'
              in kadaneHelper (a .: as) meh msf
              =: kadaneHelper as meh' msf'
              ?? ih `at` (Inst @"meh" meh', Inst @"msf" msf')
              =: msf' `smax` mss as `smax` (meh' + mssBegin as)
              =: qed

  calc "kadaneCorrect" (\(Forall xs) -> kadane xs .== mss xs) $
       \xs -> let z = 0 :: SInteger
              in [] |- kadane xs
                    =: kadaneHelper xs z z
                    ?? inv `at` (Inst @"xs" xs, Inst @"meh" z, Inst @"msf" z
    )
                    =: z `smax` mss xs `smax` (z + mssBegin xs)
                    =: mss xs `smax` mssBegin xs
                    =: mss xs
                    =: qed
```

The inductive step unfolds `kadaneHelper` once and applies the hypothesis at the new accumulators. Here the hint is more specific than in the earlier proofs: `ih `at` (Inst @"meh" meh', Inst @"msf" msf')` instantiates the quantified accumulators explicitly. The solver could search for the instantiation, but stating it makes the step easier and documents the argument. The remaining equality between the instantiated hypothesis and the invariant for `a .: as` is arithmetic over `smax`, which the solver checks without further help, using the definitions of `mss` and `mssBegin`. The preconditions are needed to instantiate the hypothesis: the new accumulators satisfy them because `meh'` is a maximum with 0 and `msf'` is a maximum with `meh'`.

The main theorem then follows by a calculation without induction. With both accumulators 0, the invariant gives $\max(0, mss(xs), mssBegin(xs))$, and the solver discharges the last two steps: `mss` is nonnegative and at least `mssBegin` by its definition. Abbreviated output shows the two results:

```
ghci> runTP kadaneCorrect
Inductive lemma: kadaneInvariant
  ...
  Result:                          Q.E.D.
Lemma: kadaneCorrect
  ...
  Result:                          Q.E.D.
```

16.5 Lessons

The effort in this proof went into one place: the invariant. Once we had explained *why* the algorithm works, as the three-way case analysis on where the best segment lies, the mechanical checking was immediate. This division of labor is typical of TP. The solver handles unfolding definitions, arithmetic, and case analysis on `smax`; the human supplies generalizations and intermediate statements. When a proof gets stuck, the useful question is rarely "which tactic is missing?" and usually "what is the statement that explains this step?"

Exercise 6. *Challenge.* Require the chosen segment to be nonempty. Write a specification and a linear-time algorithm, and prove their equivalence under an explicit nonempty-input precondition. Test all-negative lists. Solutions: `mssNonEmpty`, `kadaneNonEmpty`, and `kadaneNonEmptyCorrect`. The independent segment enumerator is `segmentNonEmpty Oracle`. The solutions return 0 on `[]`, outside the theorem's contract.

17 Induction on decreasing measures

The next two examples use `sInduct` to choose a measure suited to the computation: the size of an expression tree, then the length of a search interval.

17.1 Expression trees

Return to `Expr`: mirroring swaps the children at every addition and multiplication node, preserving evaluation because both operations commute.

```
mirror :: SExpr -> SExpr
mirror = smtFunction "mirror" $ \e -> [sCase| e of
  Num n   -> sNum n
  Add a b -> sAdd (mirror b) (mirror a)
  Mul a b -> sMul (mirror b) (mirror a)
  |]
```

Each binary case needs the hypothesis for both children before commutativity applies. The calculational `induct` interface supports integers and lists, so for this datatype we use `sInduct` with a size measure. Define `exprSize` to be zero at `Num` and one plus the two child sizes at `Add` or `Mul`. The companion proves nonnegativity as `exprSizePos`, using `inductiveLemma`: it applies the generated datatype induction principle without further guidance. Then `exprSizeSmaller` uses this fact to show that each child is smaller than its parent. These facts justify applying the hypothesis to each child. The listing abbreviates the identical multiplication case; the companion checks all three cases in full:

```
mirrorCorrect :: TP (Proof (Forall "e" Expr -> SBool))
```

```
mirrorCorrect = do
  pos <- recall exprSizePos
  smaller <- recall exprSizeSmaller
  sInduct "mirrorCorrect" (\(Forall e) -> eval (mirror e) .== eval e)
    (exprSize, [proofOf pos]) $ \ih e -> [] |- [pCase| e of
      Num n -> eval (mirror (sNum n)) =: eval (sNum n) =: qed
      Add a b -> eval (mirror (sAdd a b))
              =: eval (mirror b) + eval (mirror a)
              ?? smaller `at` (Inst @"a" a, Inst @"b" b)
              ?? ih `at` Inst @"e" a
              ?? ih `at` Inst @"e" b
              =: eval b + eval a
              =: eval a + eval b
              =: eval (sAdd a b)
              =: eval e
              =: qed
      Mul a b -> -- as for Add, with (*) in place of (+)
      |]
```

The hypothesis supplied by `sInduct` is conditional: it proves the result for an expression only if its measure is smaller than that of the current `e`. Each `smaller` hint discharges these conditions; the two `ih` hints instantiate the hypothesis separately at `a` and `b`. After substituting their evaluations, commutativity finishes the case. The final `eval e` reconnects each binary case to the theorem's goal. The `pCase` quasi-quoter splits the *proof* by constructor and checks that the cases cover the datatype; `sCase` defines symbolic functions. The `Num` case has no recursive children and needs no hypothesis. Running `runTP mirrorCorrect` checks all three cases and returns `Proven`. The size argument is structural induction expressed through `sInduct`'s more general principle of induction on a decreasing measure.

17.2 Binary search with a decreasing interval

Section 3.2 checked midpoint arithmetic. We now ask whether the search itself finds a target in a sorted interval. SBV's `TP.BinarySearch` example uses `Arr = SArray Integer Integer` and `Idx = (SInteger, SInteger)`, with both bounds inclusive:

```
bsearch :: Arr -> Idx -> SInteger -> SMaybe Integer
bsearch array (low, high) = f array low high
  where f = smtFunctionWithMeasure "bsearch" (\_arr lo hi _x -> (hi - lo + 1) `smax`
      0, [])
          $ \arr lo hi x ->
               let mid  = (lo + hi) `sEDiv` 2
                   xmid = arr `readArray` mid
               in [sCase| lo of
                     _ | lo .> hi   -> sNothing
                     _ | xmid .== x -> sJust mid
                     _ | xmid .< x  -> bsearch arr (mid+1, hi)    x
                     _              -> bsearch arr (lo,    mid-1) x
                  |]
```

Here `sEDiv` by 2 computes the floor of the integer average, including for negative bounds. The measure $\max(hi - lo + 1, 0)$ is the number of indices remaining. On a nonempty interval,

the midpoint lies within its bounds, so either recursive call strictly decreases this measure. The empty interval returns `sNothing`; a match returns `sJust mid`.

The companion imports this module as `Search` and `Data.SBV.Maybe` as `SM`. We state the result contract explicitly:

```
searchFound :: Search.Arr -> Search.Idx -> SInteger
            -> SMaybe Integer -> SBool
searchFound arr (lo, hi) x result =
  SM.isJust result .&& lo .<= i .&& i .<= hi .&& readArray arr i .== x
  where i = SM.fromJust result

searchContract :: Search.Arr -> Search.Idx -> SInteger
               -> SMaybe Integer -> SBool
searchContract arr bounds x result =
  ite (Search.inArray arr bounds x)
      (searchFound arr bounds x result)
      (SM.isNothing result)
```

The predicate `Search.inArray` means that the target occurs at some index between the bounds. If it occurs, `searchFound` requires a `Just` result, an index within the interval, and the correct value at that index. Otherwise the result must be `Nothing`. The theorem assumes that the array is nondecreasing throughout the interval; duplicates are allowed, and no particular matching index is prescribed.

The proof `binarySearchCorrect` uses `sInduct` with the same interval measure. Its two helper lemmas show that restricting the search to the selected half preserves sortedness and whether the target occurs. In the upper-half case, for example, all values at or below the midpoint are too small. The calculation supplies that range lemma and instantiates the induction hypothesis at $[mid + 1, hi]$; the lower-half case is symmetric. Empty intervals and immediate matches close directly. Running `binarySearchProof` checks this argument with CVC5, including termination of `bsearch`; the complete calculation is in the companion.

This proof uses unbounded integer indices, so `lo + hi` cannot overflow. It complements the earlier arithmetic proof, but does not verify a fixed-width implementation of the whole search: that further step must also justify the index conversions and updates `mid+1` and `mid-1`.

18 State transitions and inductive invariants

18.1 Transition systems and bounded model checking

18.2 Inductive invariants and strengthening

18.3 Weakest preconditions and imperative programs

19 Polymorphic and higher-order proofs

Proof scripts can be reused across types and parameterized by operations. The next examples show what each form of reuse establishes.

19.1 Polymorphic scripts, monomorphic proofs

Reversal preserves length regardless of the element type. We can express the proof once, using Haskell polymorphism:

```
reverseLength :: forall a. SymVal a => TP (Proof (Forall "xs" [a] -> SBool)
    )
reverseLength = induct "reverseLength"
  (\(Forall xs) -> SL.length (SL.reverse xs) .== SL.length xs) $
  \ih (x, xs) -> [] |- SL.length (SL.reverse (x .: xs))
                    =: SL.length (SL.reverse xs SL.++ [x])
                    =: SL.length (SL.reverse xs) + 1
                    ?? ih
                    =: SL.length xs + 1
                    =: SL.length (x .: xs)
                    =: qed
```

The Haskell `forall a` makes the *script* polymorphic; `SymVal a` provides the symbolic representation of its elements. Before running it, we choose a monotype. For example, in GHCi:

```
runTP (reverseLength @Integer)
runTP (reverseLength @Word8)
```

Each invocation checks a separate theorem over that type's solver sort. The companion runs both.

This differs from `Forall "xs" [a]`, which quantifies over *values* inside the theorem after `a` has been chosen. A successful integer instance does not by itself certify every type instance of the script: SBV is not quantifying over Haskell types. Polymorphism lets us reuse the argument and its implementation, while each proof run establishes a monomorphic result. This distinction matters especially for arithmetic laws, whose truth may change between `Integer` and `Word8`.

19.2 The bookkeeping law

Many useful list laws concern an operation supplied by the caller. The *bookkeeping law*, adapted here from `Documentation.SBV.Examples.TP.Lists`, says that reducing a concatenation equals reducing each chunk and then reducing those results:

$$reduce(concat\ xss) = reduce(map\ reduce\ xss), \qquad reduce = foldr\ f\ e.$$

Associativity of f and a two-sided identity e suffice. For integer addition with identity 0, this is "sum the elements" versus "sum the chunk sums." Commutativity is not assumed: the order of elements is preserved. Both the outer list `xss` and every chunk are finite; the law makes no claim about consuming an infinite stream of chunks.

We reuse a library lemma with the following import:

```
import qualified Documentation.SBV.Examples.TP.Lists as ListProofs
```

The proof has two dependencies: `foldrOverAppend`, and a helper `foldBase` proved in the companion. Their conclusions are, respectively,

$$foldr\ f\ e\ (xs\ ++\ ys) = foldr\ f\ (foldr\ f\ e\ ys)\ xs,$$

$$foldr\ f\ y\ xs = f\ (foldr\ f\ e\ xs)\ y.$$

The first needs no assumptions on f. The second requires associativity and the left-identity law. Its empty-list case uses $f\ e\ y = y$; its inductive step uses associativity to move y past the head. These lemmas expose exactly the two transformations the main induction needs:

```
bookKeeping :: forall a. SymVal a
            => SBV a -> (SBV a -> SBV a -> SBV a)
            -> TP (Proof (Forall "xss" [[a]] -> SBool))
bookKeeping e f = do
  let assoc = quantifiedBool $ \(Forall x) (Forall y) (Forall z) ->
                f x (f y z) .== f (f x y) z
      rUnit = quantifiedBool $ \(Forall x) -> f x e .== x
      lUnit = quantifiedBool $ \(Forall x) -> f e x .== x
      reduce = SL.foldr f e
  base <- foldBase e f
  over <- ListProofs.foldrOverAppend e f
  induct "bookKeeping"
    (\(Forall xss) -> assoc .&& rUnit .&& lUnit .=>
       reduce (SL.concat xss) .== reduce (SL.map reduce xss)) $
    \ih (xs, xss) -> [assoc, rUnit, lUnit] |- reduce (SL.concat (xs .: xss))
    )
      =: reduce (xs SL.++ SL.concat xss)
      ?? over
      =: SL.foldr f (reduce (SL.concat xss)) xs
      ?? ih
      =: SL.foldr f (reduce (SL.map reduce xss)) xs
      ?? base `at` (Inst @"xs" xs, Inst @"y" (reduce (SL.map reduce xss)))
      =: f (reduce xs) (reduce (SL.map reduce xss))
      =: reduce (reduce xs .: SL.map reduce xss)
      =: reduce (SL.map reduce (xs .: xss))
      =: qed
```

`quantifiedBool` packages a quantified property as an `SBool`, so the algebraic laws can appear as assumptions in the calculation and as the premise of the theorem. They are not unconditionally added as user axioms. Induction is over the outer list of chunks: unfold `concat`, move the fold across append, use the induction hypothesis on the remaining chunks, then use `foldBase` to combine their result with that of the first chunk.

The left-identity assumption has a concrete purpose. Take $f\ x\ y = x$ and $e = 0$. This operation is associative and has right identity 0, but for `xss = [[],[1]]`, the two sides of the bookkeeping law are 1 and 0. The empty first chunk contributes an identity that must work on the left. The companion checks this counterexample as well as the proof.

19.3 An arbitrary operation at a fixed type

The parameters `e` and `f` are Haskell arguments to the proof script. To prove the law without choosing an implementation of the operation, give them fresh uninterpreted names:

```
bookKeepingAbstract :: IO (Proof (Forall "xss" [[Integer]] -> SBool))
bookKeepingAbstract = runTP $
  bookKeeping @Integer (uninterpret "unit") (uninterpret "combine")
```

An `uninterpret` declaration supplies a logical name without defining equations. Here `combine` is a total function on integers; the proof must work for every interpretation satisfying the stated algebraic laws. The solver does not enumerate implementations of `combine`. SBV specializes its list operations to this logical operation and reasons about the resulting definitions. The higher-order Haskell interface thus supports a proof about an arbitrary operation at a fixed type. We do not pass a Haskell function as an ordinary `Forall` argument of the theorem.

Passing `0` and `(+)` instead gives a concrete instance. The companion also checks that instance and derives the unconditional sum-of-chunk-sums law. An uninterpreted operation represents a total mathematical function; it does not model divergence or exceptions in a Haskell callback.

19.4 Closures and higher-order functions

Ordinary Haskell allows `map (+k) xs`; `SL.map (+k) xs` needs an explicit `Closure` when `k` is a symbolic unknown from the enclosing solver context:

```
shiftEach :: SInteger -> SList Integer -> SList Integer
shiftEach k = SL.map Closure { closureEnv = k
                             , closureFun = \env x -> x + env
                             }
```

SBV passes `k` through the recursion as `env`, so `closureFun` need not capture it. Literals and globally named logical symbols need no such environment. The companion proves length preservation for every offset and finite integer list.

20 A verified tautology checker

Larger developments are built from the same ingredients. SBV's documentation includes a verified tautology checker for propositional formulas in if-then-else form, after a classic development by Boyer and Moore (1979, Chapter 4) that was later formalized in Imandra (Passmore nd). We sketch its structure rather than its proofs; the full development is in the module `Documentation.SBV.Examples.TP.TautologyChecker`. The listings in this section are quoted from that module.

Formulas and normalization. A formula is a constant, a variable, or a conditional:

```
data Formula = FTrue
             | FFalse
             | Var { fVar   :: Integer }
             | If  { ifCond :: Formula
                   , ifThen :: Formula
                   , ifElse :: Formula
                   }
```

The checker first normalizes a formula so that no conditional appears in the condition position of another conditional, by repeatedly rewriting `If (If p q r) l r'` to `If p (If q l r') (If r l r')`. Abbreviated slightly, the definition is:

```
normalize :: SFormula -> SFormula
normalize = smtFunctionWithMeasure "normalize"
    ( \f -> tuple (ifComplexity f, ifDepth f)
    , [measureLemma ifDepthNonNeg, measureLemma ifComplexityPos, ...] )
  $ \f -> [sCase| f of
             If (If p q r) l r' -> normalize (sIf p (sIf q l r') (sIf r l r
    '))
             If c l r           -> sIf c (normalize l) (normalize r)
             _                  -> f
          |]
```

This function is the first new ingredient. Its termination is not structural: the rewrite in the first branch produces a *larger* formula. Instead of letting SBV guess a measure, the definition supplies one through `smtFunctionWithMeasure`: a pair ordered lexicographically. To see why this works, write C for `ifComplexity` and D for `ifDepth`. On constants and variables, $C = 1$ and $D = 0$; on conditionals,

$$C(\mathit{If}\ c\ l\ r) = C(c)\big(C(l) + C(r)\big), \qquad D(\mathit{If}\ c\ l\ r) = 1 + D(c).$$

The distributive rewrite preserves C exactly and reduces D by one. Recursing into either branch of an ordinary conditional strictly reduces C, because every complexity is positive. Thus each recursive call decreases the pair, even though tree size may increase. These checks use proved facts supplied by `measureLemma`, including positivity of C. Termination and truth preservation remain separate obligations.

Checking. On a normalized formula, the checker walks down the conditionals, recording an assumed value for each variable the first time it is tested. When it meets a variable that was already assigned, it follows the corresponding branch; otherwise it checks both branches, once under each assumption. The function `isTautology f` normalizes `f` and runs this walk with no assumptions.

The theorems. Soundness says that a formula accepted by the checker evaluates to true under every assignment:

```
soundness :: TP (Proof (Forall "f" Formula -> Forall "bindings" [Binding] -> SBool))
soundness = do
  tie <- recallWith cvc5 tautologyImpliesEval
  nrt <- recall normalizeRespectsTruth
  nc  <- recall normalizeCorrect

  calc "soundness"
       (\(Forall f) (Forall bindings) -> isTautology f .=> eval f bindings) $
       \f bindings -> [isTautology f]
                   |- eval f bindings
                   ?? nrt `at` (Inst @"f" f, Inst @"bs" bindings)
                   =: eval (normalize f) bindings
                   ?? nc  `at` Inst @"f" f
                   ?? tie `at` (Inst @"f" (normalize f), Inst @"a" bindings, Inst @"
    b" [])
                   =: sTrue
                   =: qed
```

Completeness says that if the checker rejects a formula, a companion function `falsify` produces an assignment under which the formula is false. Both top-level proofs are short because the work is in their lemmas. The development contains 21 TP lemmas. Several prove properties of variable lookup when assignments are extended; others establish that normalization produces a normal formula and preserves meaning. The central inductive lemmas, such as `tautologyImpliesEval`, use *strong induction* via `sInduct`: the hypothesis is available for every formula with a smaller measure, not only for immediate subformulas, which matches the shape of the recursion in `isTautology'`. The lemmas use `recall` to reuse previously proved results; some obligations use CVC5. The companion checks both top-level theorems and their dependencies.

What to take away. The development reuses every technique from the earlier sections at larger scale. Generalization appears again: the central lemma `tautologyImpliesEval` is stated for arbitrary variable assignments, not only for the empty list of assumptions with which `isTautology` starts. Here strong induction extends beyond immediate children, and termination uses an explicit lexicographic measure. The structure of the proof follows the structure of the program: one group of lemmas per auxiliary function, and top-level theorems assembled from them by short calculations. SBV's documentation contains about forty further TP developments, including sorting algorithms, a semantics-preserving constant folder, and a compiler from expressions to a stack machine.

21 C generation from verified functions

21.1 A verified function across the translation boundary

A theorem about a symbolic function and a claim about an executable program meet at a translation boundary. To make that boundary concrete, we return to the first specification in the tutorial. SBV translates a symbolic function to C code that runs without a solver. For the saturating adder of Section 3:

```
generateAdder :: FilePath -> IO ()
generateAdder dir = compileToC (Just dir) "sat_add" $ do
  cgOverwriteFiles True
  cgGenerateDriver False
  x <- cgInput "x"
  y <- cgInput "y"
  cgReturn $ satAdd x y
```

This produces `sat_add.c` and a matching header. Because the C code is generated from the same `satAdd` that we proved correct, there is no manual rewrite in which the two could drift apart. For this adder, the generated header maps `SWord8` to `uint8_t`, and the arithmetic helpers implement 8-bit modular addition and subtraction using unsigned intermediates and masks.

21.2 Executable validation and trust assumptions

The companion checks the compiled artifact exhaustively: it compiles the generated adder with undefined-behavior sanitization and compares all 65,536 inputs with an independent oracle written directly in C. This checks the translation and compilation boundary for the generated executable; Section 23 explains the remaining trust assumptions.

Not all of SBV's vocabulary is executable. Quantifiers and uninterpreted sorts, for example, have no operational meaning, and the code generator rejects programs that use them rather than approximating them. Uninterpreted functions become calls to C functions that the user provides, which then join the trusted code.

22 The execution model and its proof obligations

Two questions connect the examples to their implementation: what does Haskell evaluate before the solver runs, and which definitions may the proof rely on? The representation of symbolic values explains the first; termination and the choice of a finite, total domain explain the second. These distinctions help predict both the cost of constructing a query and the scope of its answer.

Symbolic values are deferred graph nodes. Internally, a value of type `SBV a` is either a concrete constant or a deferred computation that, given the state of the current symbolic run, adds a node to a graph and returns its identifier:

```
newtype SBV a = SBV SVal
data SVal     = SVal Kind (Either CV (Cached SV))
newtype Cached a = Cached (State -> IO a)
```

The addition function `\x y -> x + y` can therefore have the pure type `SWord8 -> SWord8 -> SWord8` and still allocate graph nodes. Operations on constants are folded immediately, which is why `satAdd 250 10` evaluates to `255` without a solver. The run-time `Kind` records the logical sort, so simplifications can respect it: rewriting `x .== x` to true is valid for integers but not for floats.

Sharing comes in two forms. A generator that combines a term with itself, such as `iterate (\t -> t + t) x !! 30`, describes a term that is exponentially large as a tree but linear as a graph. SBV preserves this sharing in two ways. It memoizes each deferred computation, using stable names in the style of observable sharing (Gill 2009), so that forcing the same Haskell value twice yields the same node. It also interns nodes, so that separately constructed applications of the same operator to the same arguments reuse a node. Neither mechanism identifies arbitrary mathematically equal expressions.

Nodes belong to a run. A graph node is meaningful only in the run that created it. Two values of type `SInteger` from different calls to `prove` cannot be combined, even though their types agree, and SBV checks for common mistakes of this kind. To carry a result from one run to another, extract a concrete value from a model and turn it back into a literal. Section 9 illustrates the same concrete-to-symbolic conversion with `literal v`, though its loop stays within one run. Constants and reusable function definitions can be shared across runs; session-specific symbolic unknowns cannot.

Queries synchronize dependencies. When a query creates a new term, SBV records the declarations it requires: sorts for datatypes (including those nested inside lists, tuples, or other datatypes), uninterpreted constants, and definitions of `smtFunctions` it calls. Before the term is used, any of these that the solver has not seen are sent. This is why a datatype can first appear inside a query. The companion exercises this case with two fresh values of a parameterized datatype created by `freshVar`.

22.1 Termination is a proof obligation

SMT functions denote total functions. An equation such as $b(n) = b(n) + 1$ has no integer-valued solution; admitting it as a definition would make the logical context inconsistent and allow unrelated claims to be proved. TP therefore checks termination of the symbolic functions used in a proof, separately from checking the theorem itself.

SBV tries automatic measures such as list length or datatype size. For recursion on a list's tail, length decreases at each call and is bounded below by zero, so the calls cannot continue indefinitely. The `Functions proven terminating` line reports these checks. If the automatic check fails, supply a measure with `smtFunctionWithMeasure`; the normalizer in Section 20 illustrates a lexicographic measure. An unknown solver result does not establish termination. Explicitly bypassing a check with `smtFunctionNoTermination` leaves a dependency reported as `Modulo` the function's termination. The companion checks that $b(n) = b(n) + 1$ is rejected.

22.2 Finite lists and datatypes

Symbolic lists and recursive algebraic datatypes contain finite, total values: no undefined elements, partial tails, or infinite constructor trees. Sizes are unbounded, but each value is finite. For example, double reversal is the identity on finite lists, while for `xs = repeat 0`, `take 1 xs` returns `[0]` and `take 1 (reverse (reverse xs))` diverges. This domain makes structural induction and decreasing-size arguments available; total reasoning and its relationship to partial languages are studied by Danielsson et al. (2006). Arrays instead denote total mappings over possibly infinite index domains (Section 12); they do not introduce infinite recursive constructor structure.

23 Proof results, diagnostics, and trust

23.1 Failed checks and diagnostic steps

Before proving, run `testSatAdd` to check `satAddSpec` on random byte inputs with `sbvQuickCheck`. The companion checks this result before proving; passing random tests is not a proof.

Pass `"sat-add.smt2"` to this helper to inspect solver commands:

```
inspectSatAdd :: FilePath -> IO ThmResult
inspectSatAdd path = proveWith
  z3{verbose = True, transcript = Just path} satAddSpec
```

Here `verbose` prints abbreviated diagnostics; `transcript` writes SMT-LIB. Replay it outside GHCi with `z3 sat-add.smt2`.

The solver cannot start. Run `z3 --version` in the same shell and check that the executable is on `PATH`. Installing SBV does not install Z3.

The solver returns unknown or times out. No conclusion follows. Simplify the obligation and check whether its theories suit the solver; nonlinear arithmetic is a common obstacle. Use `z3Timeout5` from Section 8 to bound each solver call to 5,000 milliseconds. `SetTimeOut` does not limit the whole TP run, which may make many calls.

A counterexample is found. Evaluate implementation and specification on that input. The result of `wrongAdd 223 124` is 91. The specification evaluates `min 255 (223`

`+ 124 :: Integer`) to 255, which agrees with the corrected `satAdd`. Check which side disagrees with the intended behavior.

A TP calculation stalls. Find the failed step and distinguish a counterexample from unknown. Check its assumptions and the `??` hints on that equality, instantiate hypotheses explicitly, or split the step with an intermediate expression. If the recursive call changes an accumulator, generalize the statement as in Sections 15 and 16.

Common pitfalls.

- *Wrong domain.* A property proved over `SInteger` says nothing about `SInt32`, and vice versa. Choose the type that matches the code you care about. List and recursive-datatype theorems cover finite, total values only (Section 22.2).
- *Specification repeats the implementation.* Write specifications in a different style or a wider domain (Section 3).
- *Haskell `if` or recursion on symbolic data.* Use `ite` and `sCase` for symbolic choices, and `smtFunction` for recursion over symbolic structure (Section 8).
- *Contradictory assumptions.* If `constrained` assumptions are unsatisfiable, everything is provable. A quick `sat` on the assumptions alone detects this.
- *Mixing runs.* Unknowns belong to the run that created them; move concrete values instead (Section 22).

What a proof rests on. A Q.E.D. from SBV depends on the correctness of SBV's translation to SMT-LIB, the solver, and, for TP, the generation of proof obligations. TP proofs also depend on any axioms the user introduces and on any functions whose termination checks were explicitly bypassed. SBV tracks these dependencies: a proof that rests on an axiom, or on an unfinished step marked with `sorry`, is reported as `Modulo` the offending lemmas rather than as `Proven`. TP does not replay solver certificates in an independent proof kernel. Functional correctness does not establish running-time or space bounds. For generated C, the C compiler and code generator join the trusted base: Section 21's exhaustive check covers the tested executable and input domain, rather than verifying either compiler in general.

23.2 Encodings and solver selection

23.3 Model validation and proof dependencies

24 Connections to program reasoning

The examples place SBV within a broader set of approaches to reasoning about functional programs. Three questions make the connections useful: how is the program represented, how is a proof decomposed, and what does a successful check establish?

24.1 Testing and symbolic evaluation

QuickCheck (Claessen and Hughes 2000) treats a property as an executable function and searches for counterexamples by sampling inputs. Symbolic reasoning instead represents unknown inputs and asks whether a violating assignment exists. For `satAddSpec`, the independent specification serves both purposes: Section 23.1 tests it before asking the solver to prove it. The practical connection is a shared specification and concrete feedback, while the evidence differs: successful tests cover the executed inputs, and an unsatisfiability result rules out a counterexample throughout the modeled domain.

Rosette (Torlak and Bodík 2014) develops solver-aided host languages using a symbolic virtual machine, demonstrated in Racket. Grisette (Lu and Bodík 2023) brings symbolic evaluation and state merging to Haskell as a library; Smten (Uhler and Dave 2013) translates high-level symbolic computations in a Haskell-derived language to SMT queries. These approaches address a common representation problem: execution with unknown data must preserve the possible outcomes of symbolic choices. In our examples, `ite` and `sCase` make those choices explicit, while ordinary Haskell computes the query generator. The what4 library (Galois, Inc. 2026) provides typed symbolic values and solver interfaces for building verification tools. SBV's interface combines symbolic programming, model extraction, and explicit proof calculations in one library.

24.2 Equational reasoning and proof guidance

Danvy (2022) teaches equational reasoning about recursive programs in Coq using fold–unfold lemmas. A recursive definition supplies equations for its constructor cases, and controlled rewriting makes their use visible in a proof. Our mirror calculation illustrates a related teaching choice: unfold the transformation, apply the hypotheses for both children, and then use commutativity. The displayed steps explain the argument even when automation could combine several of them. The checking mechanisms differ: our `=:` links generate SMT obligations, and hints provide facts for the following link.

The expression evaluator also connects to tutorials on programming-language semantics. Hutton (2023) uses a minimal arithmetic language to develop semantic descriptions and calculate an abstract machine. Our evaluator serves as the specification of a transformation: `mirrorCorrect` proves that changing syntax preserves meaning. Reusing `Expr` for search and proof lets the reader see the change from finding one term with a desired evaluation to establishing an equation for every finite term.

Generalization is the crucial step when the computation carries extra state. The reversal proof needs a statement about an arbitrary accumulator because a recursive call changes it; Kadane's proof needs an invariant describing the remaining suffix under arbitrary admissible accumulator values. In each case, the stronger statement makes the induction step possible. This is the reusable lesson of the calculations, independently of their concrete proof notation.

24.3 Refinements, invariants, and the scope of a proof

Liquid Haskell (Vazou et al. 2014) adds logical refinements to Haskell types. Refinement reflection (Vazou et al. 2018) makes function definitions available to the refinement logic and supports equational proofs written as Haskell programs, with pattern matching and recursion expressing case splits and induction. It therefore shares our interest in combining user-written proof structure with SMT reasoning. The interface differs: Liquid Haskell checks refinements of Haskell programs, while our SBV scripts construct symbolic models and execute TP checks. A `Proof` value is obtained by running those checks; ordinary GHC typechecking of a script does not establish its theorem.

Dafny (Leino 2010) provides a language and verifier in which programmers supply specifications and invariants and the verifier uses SMT to discharge the resulting obligations. The accumulator invariant in our Kadane development plays a comparable explanatory role: it describes what remains true as the state changes, here across recursive calls rather than an imperative loop.

For readers moving between these approaches, it is useful to carry over the specification and the mathematical argument before translating the notation. Check which numeric domains, termination conditions, and notions of equality the target system uses. Our finite-data scope (Section 22.2) and monomorphic theorem instances (Section 19.1) are part of the claim being proved, not incidental details of the proof script.

25 Conclusion

Solver-aided reasoning combines executable feedback with explicit mathematical structure. The examples develop a repeatable method: choose the symbolic domain, write an independent specification, and use counterexamples to check both the code and its intended behavior. When automation stalls, expose intermediate equalities and generalize the induction hypothesis to cover changing accumulators. Check termination separately, and read each theorem within its modeled domain. The resulting progression makes both success and failure informative: a counterexample helps refine a specification, and an unresolved proof obligation can expose the need for a stronger invariant or a more explicit argument. SBV supplies an executable setting for learning these decisions alongside the programs they justify.

For further practice, three SBV example modules extend these ideas. `ConstFold` proves that constant folding preserves expression meaning; `InsertionSort` combines ordering and permutation properties; and `VM` verifies compilation to a stack machine. All three are under `Documentation.SBV.Examples.TP`.

Acknowledgments

We thank Joel Burget, Iavor Diatchki, Brian Huffman, John Matthews, Brian Schroeder, and Jeffrey Young for their contributions to SBV's design and development, and the wider SBV community for bug reports, feedback, and examples.

References

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 13243)*. Springer, Cham, Switzerland, 415–442. https://doi.org/10.1007/978-3-030-99524-9_24

Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2025. *The SMT-LIB Standard: Version 2.7*. Technical Report. Department of Computer Science, The University of Iowa. https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-07-07.pdf Release of 7 July 2025.

Jon Bentley. 1984. Programming Pearls. *Commun. ACM* 27, 9 (1984), 865–873. https://doi.org/10.1145/358234.381162

Joshua Bloch. 2006. Extra, Extra—Read All About It: Nearly All Binary Searches and Mergesorts are Broken. Google Research Blog, 2 June 2006. https://research.google/blog/extra-extra-read-all-about-it-nearly-all-binary-searches-and-mergesorts-are-broken/

Robert S. Boyer and J Strother Moore. 1979. *A Computational Logic*. Academic Press, New York. https://www.cs.utexas.edu/~boyer/acl.pdf

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 268–279. https://doi.org/10.1145/351240.351266

Nils Anders Danielsson, John Hughes, Patrik Jansson, and Jeremy Gibbons. 2006. Fast and Loose Reasoning is Morally Correct. In *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, USA, 206–217. https://doi.org/10.1145/1111037.1111056

Olivier Danvy. 2022. Fold–unfold lemmas for reasoning about recursive programs using the Coq proof assistant. *Journal of Functional Programming* 32, Article e13 (2022), 27 pages. https://doi.org/10.1017/S0956796822000107

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Levent Erkök. 2026. SBV: SMT Based Verification in Haskell. Haskell library; tutorial checked with the 14.10 source tree. https://github.com/LeventErkok/sbv

Levent Erkök and Nikolaj Bjørner. 2021. Incorrect Real Optimization Results. Z3 issue 5314 and discussion. https://github.com/Z3Prover/z3/issues/5314

Galois, Inc. 2026. what4: Solver-agnostic symbolic values support for issuing queries. Hackage package `what4`, version 1.8. https://hackage.haskell.org/package/what4-1.8

Andy Gill. 2009. Type-Safe Observable Sharing in Haskell. In *Proceedings of the Second ACM SIGPLAN Symposium on Haskell*. ACM, New York, NY, USA, 117–128. https://doi.org/10.1145/1596638.1596653

Graham Hutton. 2023. Programming language semantics: It's easy as 1,2,3. *Journal of Functional Programming* 33, Article e9 (2023), 26 pages. https://doi.org/10.1017/S0956796823000072

K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (Lecture Notes in Computer Science, Vol. 6355)*. Springer, Berlin, Heidelberg, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20

Sirui Lu and Rastislav Bodík. 2023. Grisette: Symbolic Compilation as a Functional Programming Library. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 16 (2023), 33 pages. https://doi.org/10.1145/3571209

Microsoft. n.d.a. Online Z3 Guide: Arithmetic. https://microsoft.github.io/z3guide/docs/theories/Arithmetic/ Accessed 4 November 2026.

Microsoft. n.d.b. Online Z3 Guide: Arithmetical Optimization. https://microsoft.github.io/z3guide/docs/optimization/arithmeticaloptimization/ Accessed 4 November 2026.

Grant Passmore. n.d.. Tautology Checker Example for ImandraX (`tautology.iml`). Imandra examples repository. https://github.com/imandra-ai/imandrax-examples/blob/main/src/tautology.iml Descriptive title; source file accessed 4 November 2026.

Cesare Tinelli. 2024. SMT-LIB: Arrays with Extensionality. https://smt-lib.org/theories-ArraysEx.shtml Theory ArraysEx, SMT-LIB 2.7, release of 21 July 2024.

Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 530–541. https://doi.org/10.1145/2594291.2594340

Richard Uhler and Nirav Dave. 2013. Smten: Automatic Translation of High-Level Symbolic Computations into SMT Queries. In *Computer Aided Verification*. Springer, Berlin, Heidelberg, 678–683. https://doi.org/10.1007/978-3-642-39799-8_45

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 269–282. https://doi.org/10.1145/2628136.2628161

Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (2018), 31 pages. https://doi.org/10.1145/3158141